University of Galway Programming and Tools for Artificial Intelligence

(CT5132, CT5148, CT5229) 2026/2027 Syllabus

James McDermott







Syllabus

Module co-ordinator

Dr. James McDermott, School of Computer Science

- james.mcdermott@universityofgalway.ie
- http://jmmcd.net
- Office CSB3044, top floor of CS Building.
- Office hours: time to be confirmed. Also by appointment.
- Education: University College Galway, Compaq, University of Limerick, University College Dublin, Massachusetts Institute of Technology
- Please call me James

Module codes and titles

University of Galway

- CT5132 Programming and Tools for Artificial Intelligence
- CT5148 Programming and Tools for Artificial Intelligence - Online
- CT5229 Programming and Tools for Artificial Intelligence (CA)

Module overview

This module is about programming and computational tools required for artificial intelligence. It uses the Python 3 language as the main vehicle, but also focusses on conceptual material rather than just the language itself. It moves fast through Python workings. It covers some Python libraries but not comprehensively. It discusses approaches to building re-usable, high quality code but not software engineering per se. It also visits the R language.

The module as part of Programmes

- CT5132 is a core module in MSc in Artificial Intelligence (classroom) (MScAI) and MSc in Adaptive Cybersecurity (MScAC), and optional for the MSc in Software Design and Development (MScSDD).
- CT5148 is a core module in the MSc in Artificial Intelligence Part-time (online) (MScAI Online).
- CT5229 is a core module in the Diploma in AI for Software Developers.

The syllabus and assessment will be slightly different for each module code, to accommodate programme-level differences. This document will describe all three modules including differences.

Why are we still learning programming?

- If you use AI to write code at a job, you still need to be able to read and discuss code
- You need to know what kinds of things are possible and what are not
- If you would prefer to out-source everything to AI, what value do you propose you will provide to your employer?
- Learning to code is learning to think. Boilerplate code doesn't involve much thinking, but we won't be writing much of that.
- "If you want to use AI to learn, it is the best tool ever invented for learning. If you want to use AI to not learn, it is the best tool ever invented for that too." – Zvi Mowshowitz.
- "Using ChatGPT to write your essays is like bringing a forklift into the weight room." – Ted Chiang

Module approach to teaching

Each week is centered on a problem. We develop a solution together, and the Python and the theory we need are introduced along the way.

Each week has exercises. Your solutions will form your portfolio.

Module outline week-by-week

1. What is `sqrt(2)`? And decrypting a message.
2. Can the computer predict a random keypress?
3. How much steel is in a bicycle?
4. How does a dictionary actually work?
5. *Review, catch-up, and interviewing.*
6. Get the farmer, wolf, goat and cabbage across the river.
7. How similar are two sequences?
8. How similar are two images?
9. Producing a table and plot for a paper.

Weeks 10-12. Self-paced R bootcamp where relevant. *Review, catch-up, and interviewing.*

R is a special case

R is part of the syllabus in CT5132 and CT5148, but not CT5229.

Students will cover the R material in their own time, or in ST5001 if they are taking that module. There is no R prerequisite.

Prerequisites

- At least one semester of computer programming in any language at undergraduate level, or comparable industry experience, is a prerequisite.
- Students in these three programmes usually have substantial experience in programming. Some will have studied Python or used it in daily work and will find many topics very easy. However it is expected that most students on these programmes will not have studied all of the topics we will cover in this module. Thus this module will teach Python “from scratch”, but we will move fast through introductory material to take account of students’ background.
- We will assume students are able to use the command-line interface and filesystem.
- Students who feel they lack prerequisites should contact the lecturer immediately and I will be happy to help.

Module learning outcomes

Students who complete this module will be able to:

- Read and write simple Python programs.
- Use numerical Python libraries for manipulation, input/output, and visualisation of numerical data using Numpy array types and the Numpy ecosystem.
- Use, explain, and implement essential tools for AI, such as graphs, memoisation, machine learning, combinatorial programming, dynamic programming.
- Read and write simple R/tidyverse programs for data analytics (CT5132/CT5148 only).

Assessment strategy

- **Continuous assessment: 50%**. Will include a portfolio and interview based on your development of solutions to homework problems each week. **Details will be announced via Canvas.**
- **Exam: 50%**. For CT5132/CT5148, a traditional paper exam, end-of-semester, on-campus. The exam will not cover R. **For CT5229, exam details to be confirmed.**

Important: to pass the module (CT5132 / CT5148), you must achieve at least 40% in the exam.

Feedback

After you submit an assignment, there will be a delay before you receive a grade. And we will not provide feedback on every element of the assignment. This is inevitable due to class size. Causes can also include extensions for other students due to illness, plagiarism issues, administrative issues, and workload on my side or on the TA's side.

But continuous, fast feedback is extremely important for your learning. Even though you may not receive fast feedback on your assignment, you have multiple opportunities for feedback. Each week we have exercises, and they have doctests, so you can confirm that your code works well. Almost all also have solutions, so you can check your approach versus mine. And each week we have labs where we can discuss exercises and solutions, and material from the module or elsewhere.

We can also use labs to discuss your assignment work-in-progress before submission.

AI Feedback

We also have the opportunity to use generative AI as a way of gaining feedback on your coding practice. After making an attempt at a solution, you can ask any LLM for feedback. Please report what works via the Discussion Board for the benefit of all students in the class and of future students. NB, do not confuse using AI for autonomous **feedback on your exercises** with using AI to help **write your assignments**.

Link between assessment strategies and learning outcomes

The assignments are take-home and will measure the student’s ability to interact with coding environments, generate functional code, and create associated documentation. The continuous assessment may include an in-class test during the normal class time. This will be announced well in advance with details. The exam (whether on-campus or online) is closed-book, and will assess the student’s code planning and code analysis abilities, as well as understanding of key concepts.

Textbook

Four books are recommended reading, all available for free.

- Allen B. Downey, *Think Python 3rd edition*, https://greenteapress.com/wp/think-python-3rd-edition/. Very good for learning to think like a computer scientist. Includes suggested prompts for you to ask AI assistants to clarify concepts for you, and links to Google Colab notebooks where you can try out the exercises.
- Jake Vanderplas, *A Whirlwind Tour of Python*, https://jakevdp.github.io/WhirlwindTourOfPython/, fast-paced, suitable for students who have some experience programming in another language. The link is to the official site for the book, which includes links to materials. A free (legal) pdf will be in Canvas.
- Jake Vanderplas, *Python Data Science Handbook*, https://jakevdp.github.io/PythonDataScienceHandbook/, a follow-on to his Whirlwind tour, covering Numpy, Pandas, Matplotlib, Seaborn, and Scikit-Learn. Fully readable online (CC-BY-NC-ND license).
- Hadley Wickham, *R for Data Science*, https://r4ds.hadley.nz/. Very good introduction to R and the tidyverse, by the originator of the latter. For the R Bootcamp only, ie not applicable to CT5229.

Further resources

The following books and resources are interesting for more background.

- Allen B. Downey, *Think Complexity 2nd edition*, https://greenteapress.com/wp/think-complexity-2e/. Interesting topics that both expand your mind and improve your programming skills.
- Toby Segaran, *Programming Collective Intelligence: Building Smart Web 2.0 Applications*, 2007. https://www.amazon.co.uk/Programming-Collective-Intelligence-Building-Applications/dp/0596529325. Ignore the subtitle: this book is about giving an intuitive understanding of several important machine learning algorithms based on “collective intelligence” in very accessible Python: recommender systems, genetic algorithms, clustering, etc. For our purposes, it is good programming practice.
- Hal Abelson, Jerry Sussman and Julie Sussman, *Structure and Interpretation of Computer Programs*,

  1984.https://mitpress.mit.edu/sites/default/files/sicp/index.html. It gives a deeper understanding of what programming is, how program parts can be made to fit together, and how a program is evaluated. The original version uses Scheme as the teaching language, but the lessons apply to Python.

Further resources

- Stuart Russel and Peter Norvig, *Artificial Intelligence: A Modern Approach* (AIMA) is very large and is the most-used textbook for "good old-fashioned AI", i.e. topics like planning, search, symbolic AI, but not machine learning/deep networks. It may be useful as background material. There is a Python version of the code: https://github.com/aimacode/aima-python.
- Peter Norvig, *Udacity CS212*, one of the earliest MOOCs covering data structures and algorithms in Python, and AI concepts such as constraint programming. Covers some of the AIMA material but focussing on the design of programs rather than deep AI. https://www.udacity.com/enrollment/cs212
- Wes McKinney, *Python for Data Analysis*. Focus on Numpy and Pandas libraries, by the author of Pandas.
- Al Sweigart, *Automate the Boring Stuff with Python*, https://automatetheboringstuff.com/.

Student communication and feedback

Communication to students will be via Canvas (including Announcements and Discussion Board) and email.

Students are encouraged to raise questions at all times – especially before/during/after class (CT5132), or during the weekly lab/Q&A/office hours (CT5148 and CT5229). There is also the Computer DISC (Drop-in Support Centre) as advertised on the School website.

Students should use email or the Canvas Discussion Board for asynchronous questions. In particular, students should use the Discussion Board for questions about content, regulations, assessment or other items which may be of general interest. Email, phone or a meeting in-person should be used for queries which contain any personal information.

Email correspondence should comply with standard professional email etiquette. In particular, students should send email only from their University of Galway email accounts, and should state their full name and ID in every email. Emails concerning group projects should cc all group members.

Plagiarism

Students are reminded of the University's policies on plagiarism. Plagiarism includes both deliberate and accidental re-use of material from other students in the programme, or other sources including AI sources, without appropriate citation/referencing/acknowledgement. Plagiarism and any attempted subversion of the assessment process can lead to severe penalties up to and including expulsion from the University. Information and guidelines are available from the University of Galway website:

- http://www.universityofgalway.ie/plagiarism/

AI Plagiarism

As already mentioned, AI assistants can be helpful for giving you feedback, generating correct code, and so on. You are a mature learner, so you already understand the benefit of doing some things for yourself in order to understand them, even if there are tools which can do them automatically. Each item of assessment in the module will carry a clear policy on allowed AI use. Last year, a small percentage of students received zero grades in assignments, and in some cases had to repeat the module, due to disallowed AI use or inability to explain their own code. When in doubt, please be in touch with me before doing anything that could endanger your reputation and your degree.

Good AI use

AI is used a lot in industry, and in academia. I use it, but I use it the right way for me. I don’t need to learn new details of Python or practise writing common program types. You do.

There is a big difference between using AI for feedback and using AI to do the work for you.

Delivery and session times (CT5132)

The CT5132 (classroom) module uses a **flipped classroom**. This means you are expected to consume the materials (watch videos and/or read some texts and/or try exercises) before class. Then the scheduled class is an interactive lecture and Q&A. If you don’t consume the materials in advance, of course you can still attend and participate in the class. However, you might find some of the discussion confusing. We also have one-hour labs.

Delivery and session times (CT5148, CT5229)

The CT5229 and CT5148 modules are delivered asynchronously. You consume the materials (watch videos and/or read some texts and/or try exercises) in your own time. We will hold an optional weekly live lab/Q&A/office hours. For CT5229 the time is Thursdays, 6-7pm. For CT5148 the time is Thursdays, 7-8pm. This will be in a Teams meeting, which you can access by logging in to Canvas. Online / part-time students are welcome to join our on-campus classes if they wish (see details above). Teams meetings will usually be recorded for later viewing.

Week-by-week guide

Every week starts with **one genuine problem**. We describe it, then gradually build a solution. The Python language features, libraries and theory we need are introduced when needed by the problem.

Four threads run through the whole module and are picked up again every week:

- **Testing.** From Week 1 every function we write carries **doctests** in its docstring, run with `python -m doctest`. (Industrial-strength testing is available in Python via the `pytest` library; we will not cover it.)
- **Complexity.** Introduced concretely in Week 4, revisited in Weeks 6, 7 and 8.
- **The Python data model.** Introduced in Week 4, revisited whenever we make our own objects behave like built-in ones.
- **Generators and laziness.** Introduced in Week 3, reused in Weeks 6 and 8.

Week 1: Introduction — what is sqrt(2)?

Problem. Compute the square root of 2 using nothing but `+`, `−`, `*` and `/`. Newton's algorithm: guess, average the guess with 2/guess, repeat. When do we stop? Also: **Caesar cipher**.

- Convergence and tolerance: "close enough" needs a definition. Absolute vs relative error, and the fact that floats are not real numbers.
- Checking our answer against `math.sqrt`.
- Housekeeping: the module, the assessment, and our AI-use policy.

New Python: values and variables; `int` vs `float`; arithmetic and comparison operators; `if`/`else`; `while` and `for`; `def`, arguments, default arguments (`tol=1e-10`), `return`; `abs()`; f-strings and `print`; `import math` and module namespaces; `math.isclose`. Tools: the REPL vs running a script, notebooks, and how to read a traceback. **Doctests**: writing examples in a docstring and running them.

Week 2: Can a computer predict your next “random” keypress?

Problem. The Aaronson oracle. You press `f` or `d` as randomly as you can. The machine tries to predict each press before you make it. Humans are bad at being random, and a simple model beats 50%.

- Build the game first, then a deliberately dumb model, then improve it.
- The **sliding window** of history using string slicing.
- To count what usually follows a window we have to look something up by a *piece of text* rather than by a position. That is a **dict**, a crucial data structure.
- We introduce a simple learning model as a `predict`/`update` pair.
- Keeping a running accuracyg.
- **Keeping the data**: save sessions to a `history_<id>.txt` file, and then replay them offline to compare every value of n on the same keypresses. Playing takes minutes; replaying takes milliseconds, which is what makes an experiment possible at all.

New Python: `dict` — literals, indexing, `in`, `d.get(key, default)`, `d.items()`, iterating, and missing keys; `collections.Counter` and `defaultdict`, and

Week 3: How much steel is in this bicycle?

Problem. A bill of materials: a product contains sub-assemblies, which contain sub-assemblies, which eventually contain parts. How to total the mass of steel in the whole product, and count parts by type?

- Nested dictionaries.
- **Recursion**: the shape of the code mirrors the shape of the data. Base case vs recursive case, and what happens if you forget the base case.
- The accumulator pattern.
- Comprehensions.
- **Generators and laziness**: a `walk` that gives us one part at a time instead of building the whole list first. It lets us stop at the first steel part without visiting the rest of the bicycle.
- Reading the bill of materials **in from a file** so that we can change it without changing the program.

New Python: `list` and `tuple`; nesting, indexing, iteration over containers; tuple unpacking; list and dict comprehensions; `sum`, `len`, `sorted` with `key`, and why `xs.

Week 4: How does `d[k]` actually work?

Problem. Build our own dictionary, `SimpleDict`. Start with a list of key-value pairs and a linear scan. It gets very slow but we will fix it with **hashing**.

- We have used `dict` a lot already — counting n-grams, holding a bill of materials. Now we find out what has been happening underneath.
- Why the linear scan is slow: **O(n)** per lookup, **O(1)** after hashing, **O(n^2)** if we scan for duplicates. From Allen Downey a good quote: the O(1) performance of dict is one of the minor miracles of computer science.
- Hashing: turning a key into a bucket index; what happens on a collision; why the table has to grow.
- Why lists cannot be dictionary keys but tuples can. Mutability and hashability.
- Making our object feel like a real dictionary, so that `len(d)`, `k in d`, `for k in d` and `d[k] = v` all just work.

New Python: `class`, `__init__`, `self`, methods, attributes; the **Python data model**: `__getitem__`, `__setitem__`, `__len__`, `__contains__`, `__iter__`, `__repr__`.

Week 5: Rest / catch-up / assignment week

Placeholder.

- No new material.
- Time to catch up on exercises from Weeks 1-4.
- Assignment work and submission.
- Labs and Q&A available as usual.
- Interviews.

Week 6: Get the farmer, wolf, goat and cabbage across the river

Problem. The farmer must ferry a wolf, a goat and a cabbage across a river in a boat that holds one item. The wolf eats the goat, and the goat eats the cabbage, if left alone. Find the shortest sequence of crossings.

- The graph we are searching **does not exist in memory**. We generate legal next states on demand.
- Representing a state so it can go into a `set` – hashability again.
- Breadth-first search: the frontier, the visited set, and remembering each state's parent so we can reconstruct the actual path, not just announce success.
- Depth-first search, and the correspondence between an explicit stack and recursion — back to Week 3.
- The same machinery on graphs that *are* given explicitly: adjacency as a dict, topological sort, and Dijkstra's algorithm with a priority queue.
- Complexity O(V+E); why BFS finds shortest paths and DFS does not; and the idea of bidirectional search, meeting in the middle.

Week 7: How similar are two sequences?

Problem. How many single-character edits turn “kitten” into “sitting”? (Not necessarily going via real English words.) This is **Levenshtein distance**. Then a similar question for real-valued time series. This requires **dynamic time warping**.

- Warm-up on the trap: the obvious recursive Fibonacci is correct and unusably slow. Watch it recompute the same value thousands of times.
- **Memoisation**: remember answers we have already computed. One decorator turns exponential into linear — at a cost in memory.
- Turning the recursion around: fill a table bottom-up instead, and now the base cases are the first row and column.
- Recovering the answer we actually wanted — the alignment or the edit sequence — by tracing back through the table, not just reading the final number.
- More about complexity: O(2^n) becomes O(nm) in time and space, and if we only need the score we can keep one row at a time and drop the space to O(m).

New Python: `functools.cache`/`lru_cache`; building and indexing a 2D table as a list

Week 8: How similar are two images?

Problem. A folder holds photographs, and some of them are similar or even the same picture, but some are saved at different sizes. Given one image, find the most similar others.

- Reading a photo into an array: height, width, and three colour channels.
- Summarising a photo by counting its colours into bins — first in plain Python, then in Numpy, and timing both.
- One row per photo, one column per colour bin, and the distance from one photo to all the others at once.
- Why the raw counts give the wrong answer, and why dividing by the pixel count puts it right.
- What Numpy did and did not buy us: a much smaller constant, the same amount of work.

New Python: Numpy arrays, `dtype`, `shape`, axes, slicing and fancy indexing; **broadcasting** and vectorisation; `axis=` and `keepdims=`; boolean masks and `reshape`; `argsort`; the `@` matrix product; `scipy.spatial.distance.cdist`;

Week 9: Reproducing a typical table and plot from a paper.

Problem. Take the oracle we wrote in Week 2 and find out what actually makes it good. Run a full factorial experiment — context length `n`, smoothing constant, backoff on/off, crossed with several opponents including recorded human sessions — and produce the results table and the figure.

- Designing the experiment before running it: factors, levels, repeats, response variable, and a baseline to compare against.
- Meeting the scikit-learn estimator API on `DummyClassifier` first: `fit`, `predict`, `score`. This is our baseline row, and it contains no machine learning to distract us.
- **Refactoring** our Week 2 oracle to that same interface — a pair of functions and a state dict become an object that owns its own state — at which point it drops straight into the code that just ran `DummyClassifier`. Nothing checks the type; it works because it has the right methods.
- Running the grid, in parallel, with results checkpointed to disk so that a crash at run 400 is not fatal.

Week 10: R bootcamp (self-paced)

Placeholder.

- Self-paced study, in your own time.
- CT5132 and CT5148 only; not CT5229.
- Reading: Hadley Wickham, *R for Data Science*.
- Not covered in the exam.

Week 11: R bootcamp (self-paced)

Placeholder.

- Self-paced study, in your own time.
- CT5132 and CT5148 only; not CT5229.
- Reading: Hadley Wickham, *R for Data Science*.
- Not covered in the exam.

Week 12: R bootcamp (self-paced)

Placeholder.

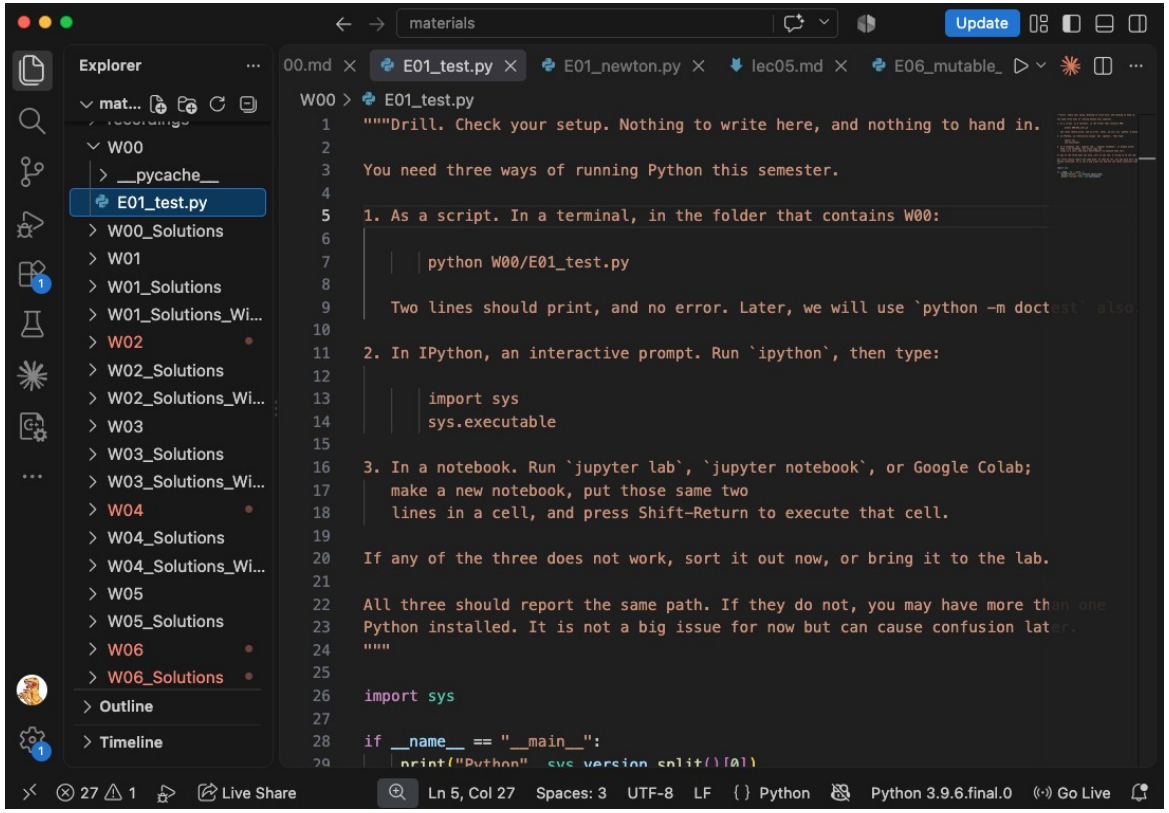
- Self-paced study, in your own time.
- CT5132 and CT5148 only; not CT5229.
- R assignment.
- Not covered in the exam.

Getting Started

Getting started with VS Code

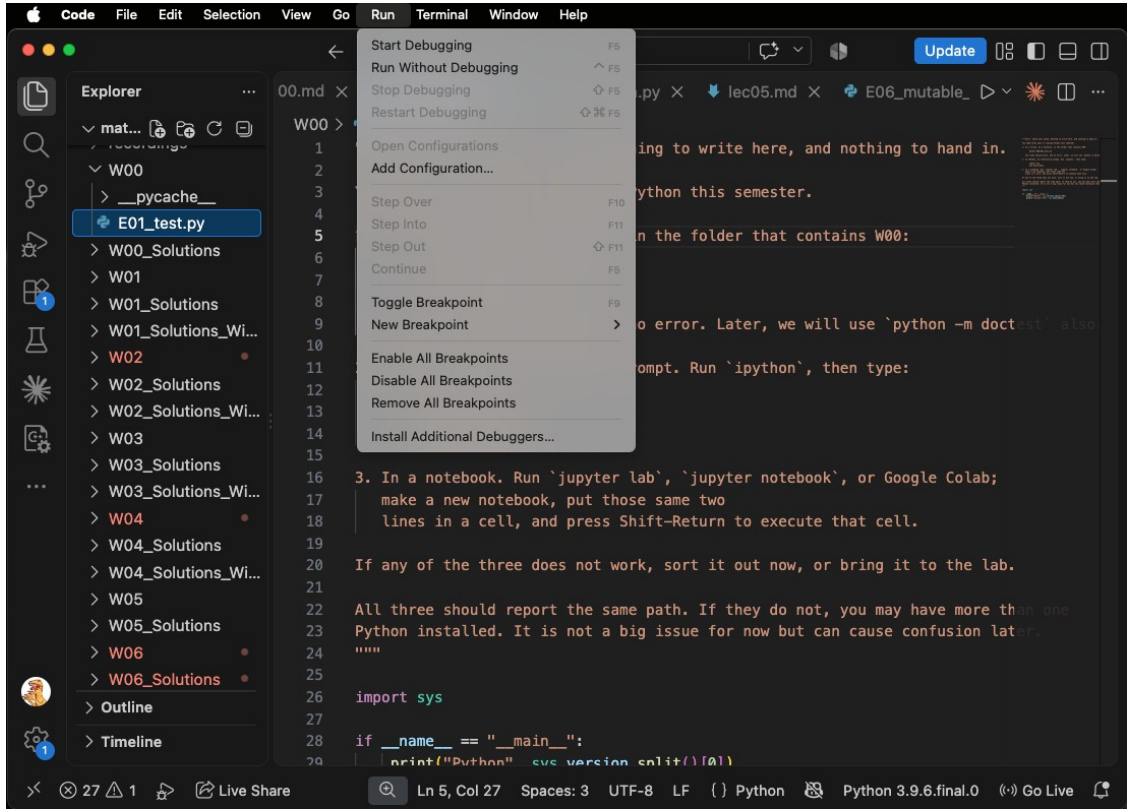
You do not have to use VS Code. But you do have to be able to run a `.py` file, and to run its doctests. The next few slides show one way that works on all three platforms.

1. Download and unzip



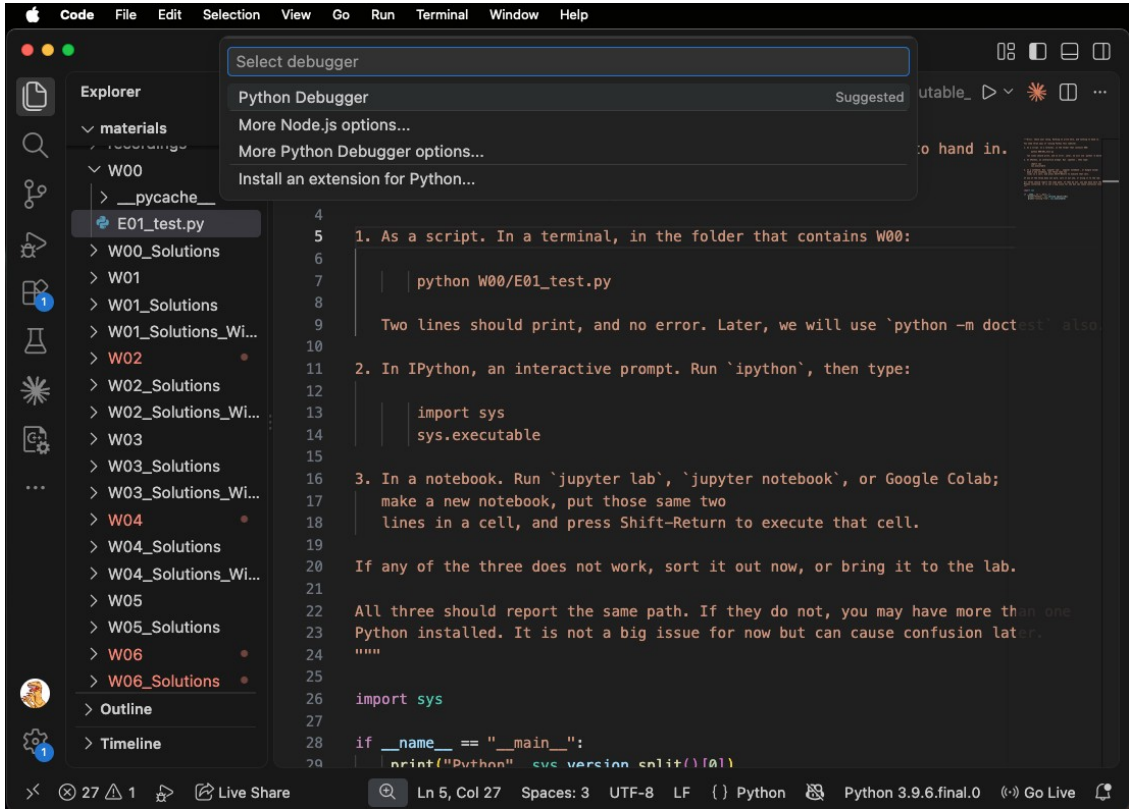
Download the week's zip from Canvas and unzip it. In VS Code, **File -> Open Folder…** and choose the folder that contains `W00`, then open `W00/E01_test.py`.

2. Run -> Run Without Debugging



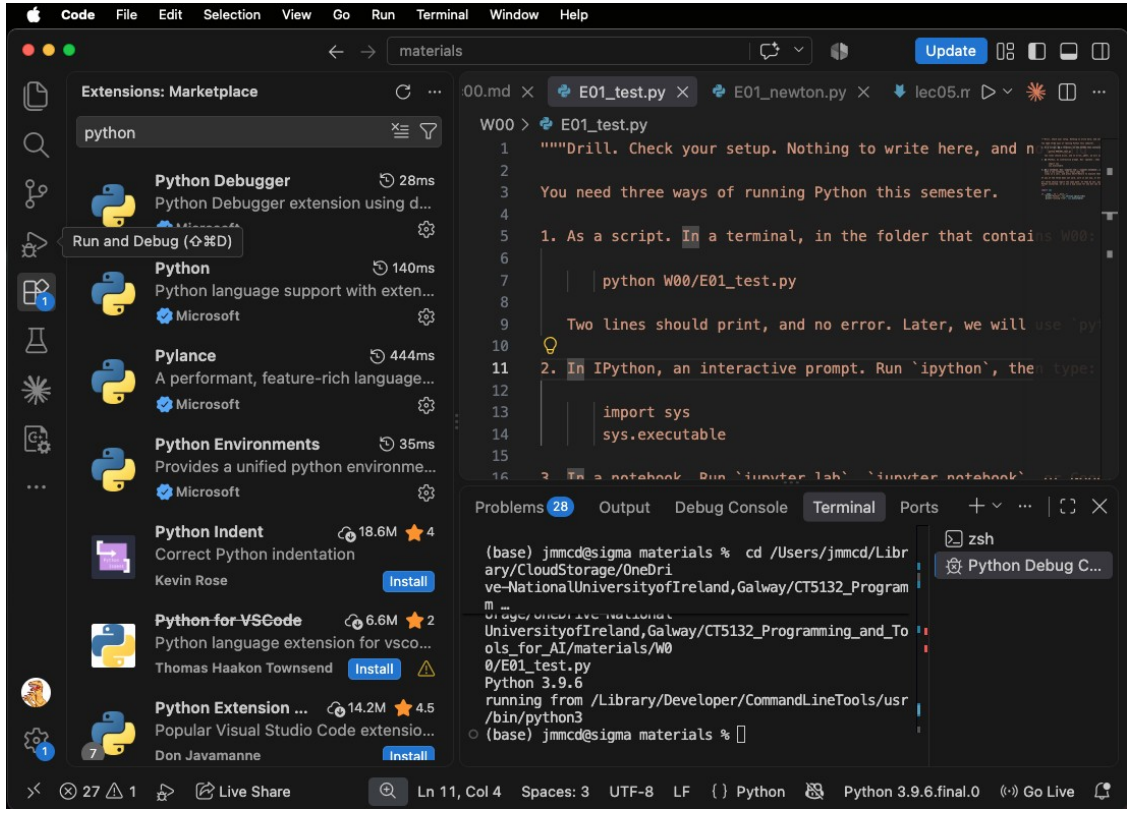
With the file open, **Run -> Run Without Debugging**.

3. Select debugger: Python Debugger



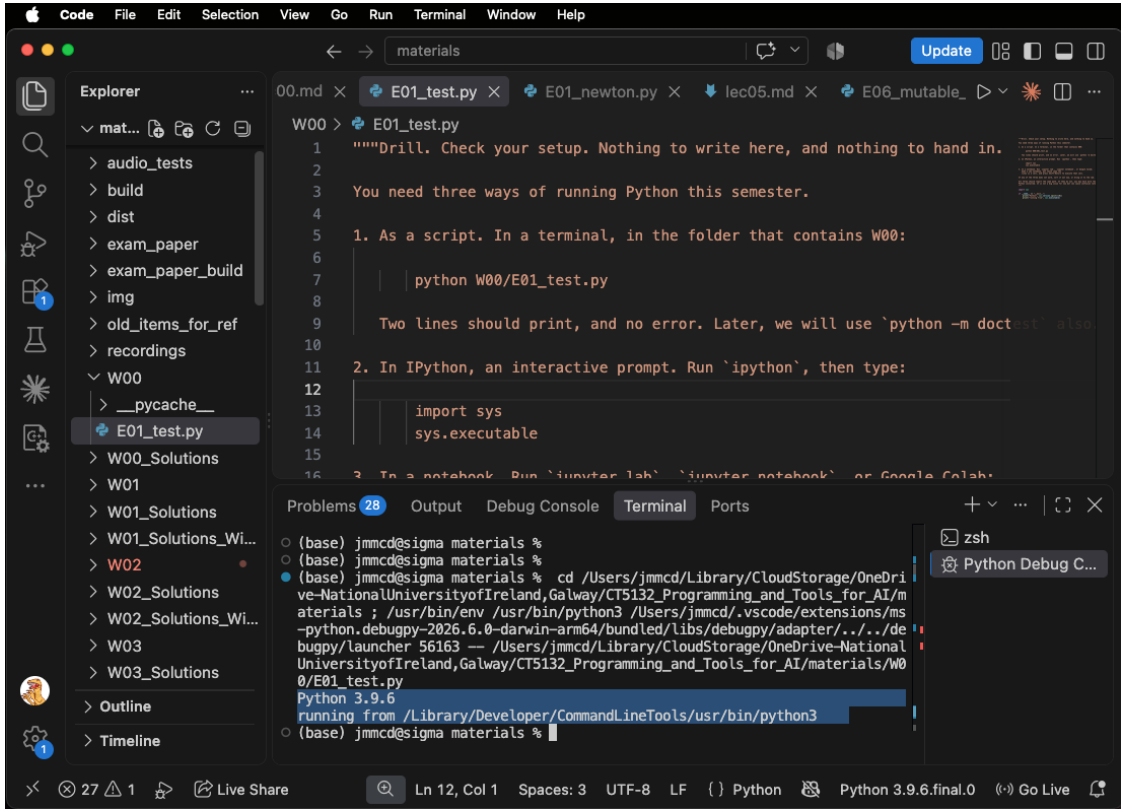
The first time only, VS Code asks what kind of file this is. Choose **Python Debugger**.

4. If Python is not offered: install the extension



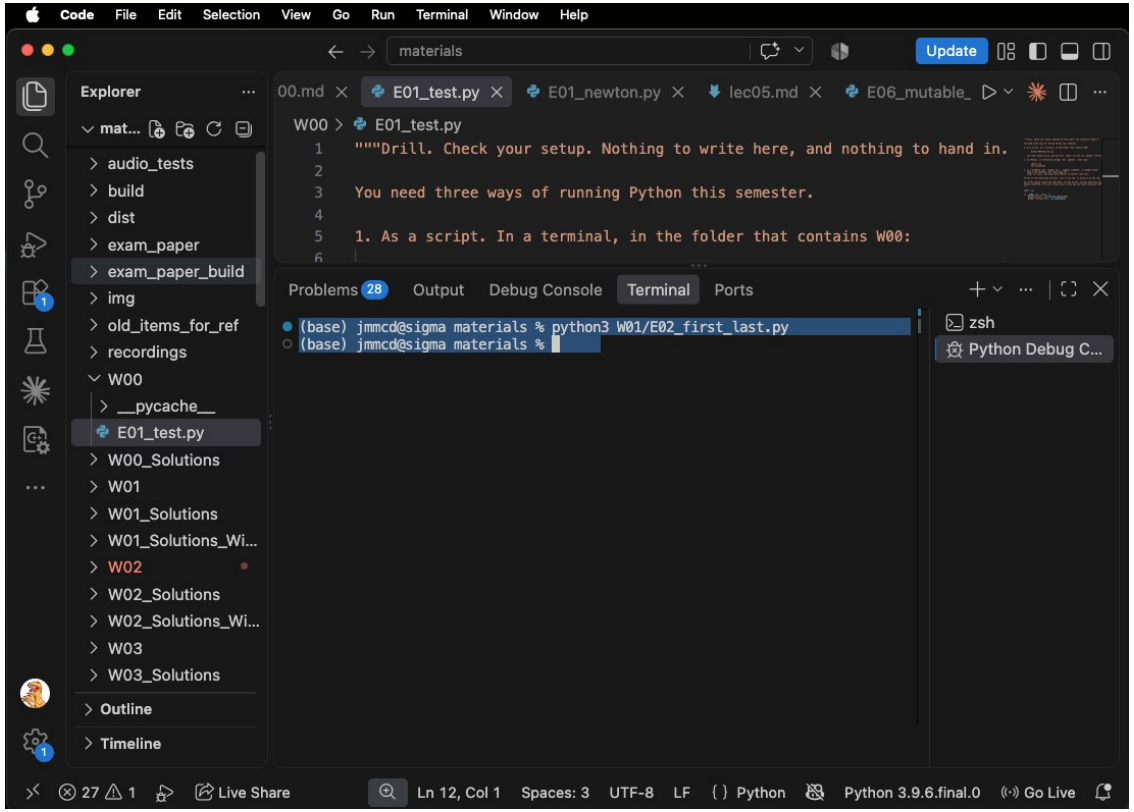
If Python is not offered, install Microsoft's **Python** extension from the Extensions panel, then try again.

5. It runs



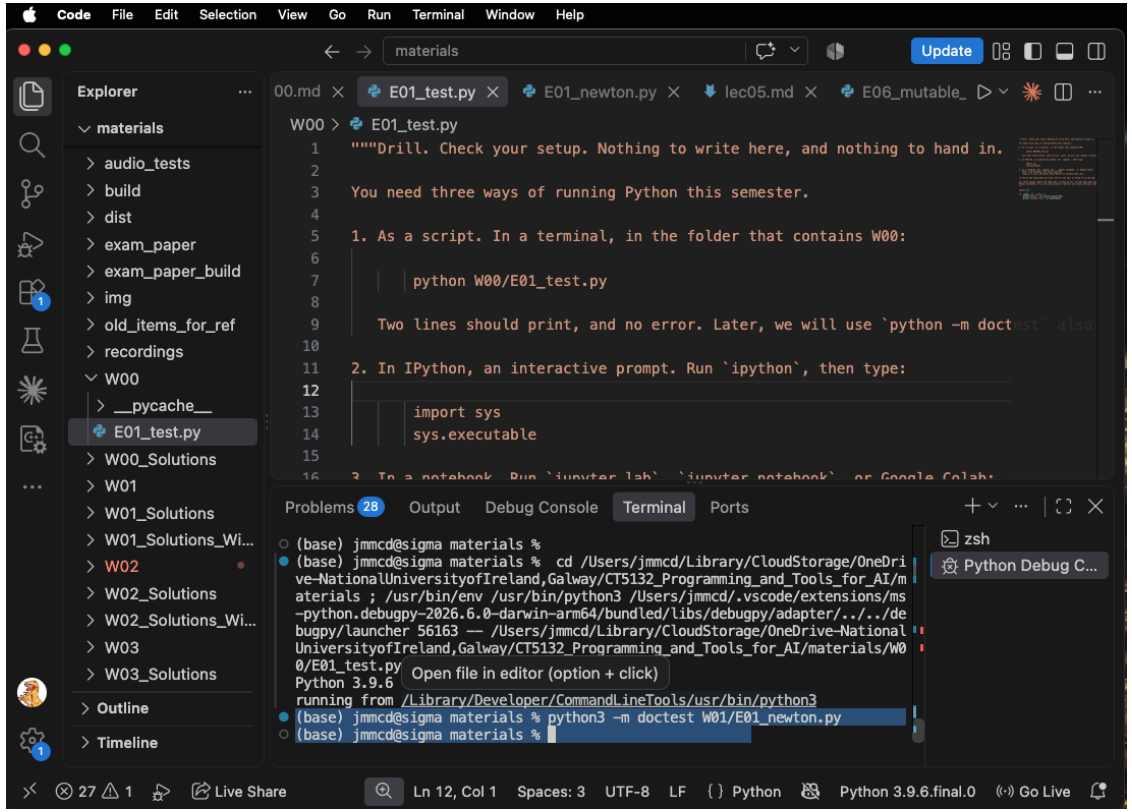
A terminal opens at the bottom, runs your file, and prints its output.

6. Running a file yourself, in the terminal



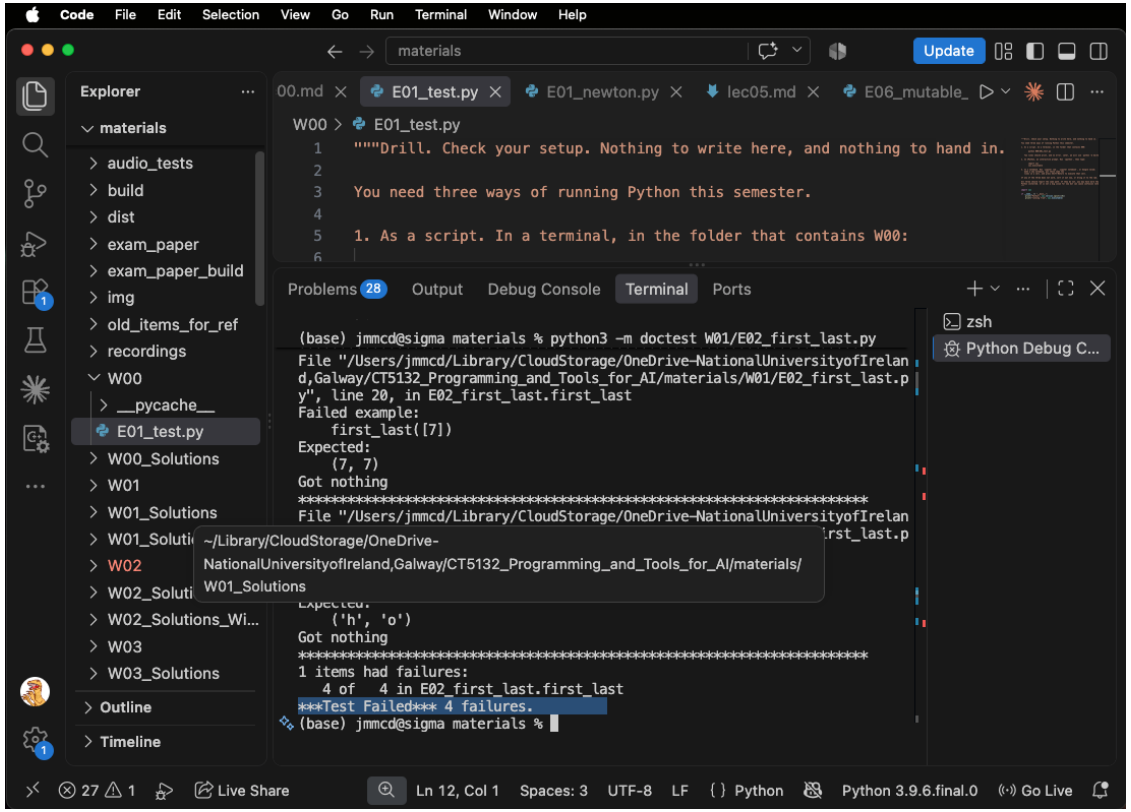
The same terminal takes commands. `python3 W01/E02_first_last.py` runs a file. This one defines a function but doesn't call it, so no output.

7. Running the doctests



`python3 -m doctest W01/E01_newton.py` runs the **tests** in the docstrings. **No output means success.**

8. What a failure looks like



The doctest run will print only the failing cases.

Summary

- Unzip the week's folder, open it in VS Code, open the `.py` file.
- **Run -> Run Without Debugging** runs the file; the first time, choose the **Python Debugger**, and install the **Python** extension if asked.
- The terminal at the bottom is the real interface: `python3 file.py` runs a file, `python3 -m doctest file.py` tests it.
- No output from doctest means every example passed. A failure prints the example, the expected answer and the actual one.

Getting Started with AI

Which AI?

Some of our assignments will have specific rules about using or not using AI for specific components. But you may use any AI model you like to help you learn.

You do not need to pay for an AI model. Two options that are free to students, and that live inside VS Code:

1. **GitHub Copilot**, using the Copilot support already built in to VS Code. Free with any GitHub account, but capped, and without the agent features. A **GitHub Education** licence lifts the cap.
2. **Google Gemini Flash**, free with a **Gemini API key**, driven by the **Cline** extension for VS Code.

Let’s set up at least one of them this week. GH takes days to be approved, so start it now and do the second one while waiting.

1. Apply for GitHub Education

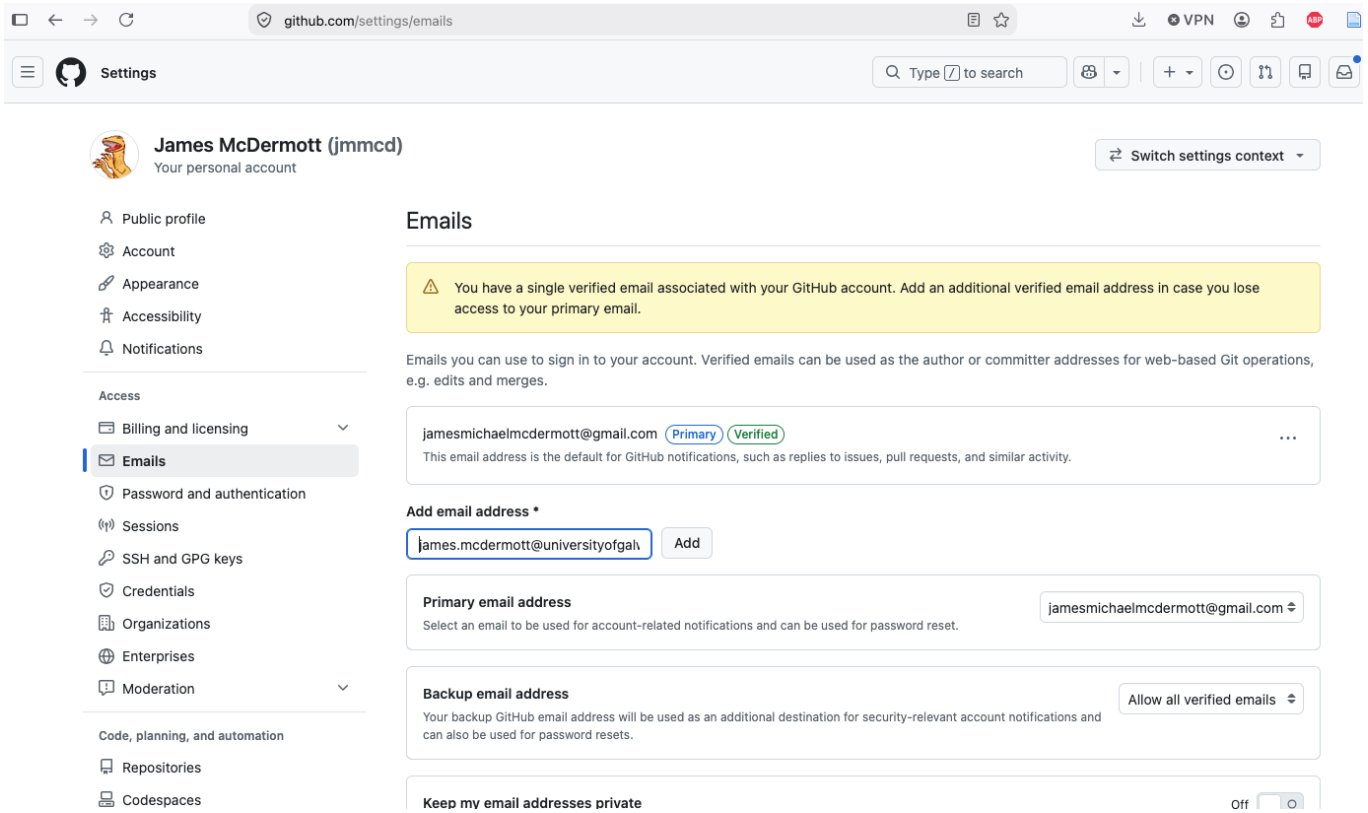
Go to **education.github.com/pack** and apply for the Student Developer Pack. You need a GitHub account first.

Approval is by a human plus some automation, and it takes anything from a few hours to a couple of weeks. **Apply today.**

Doing it quickly: the gotchas

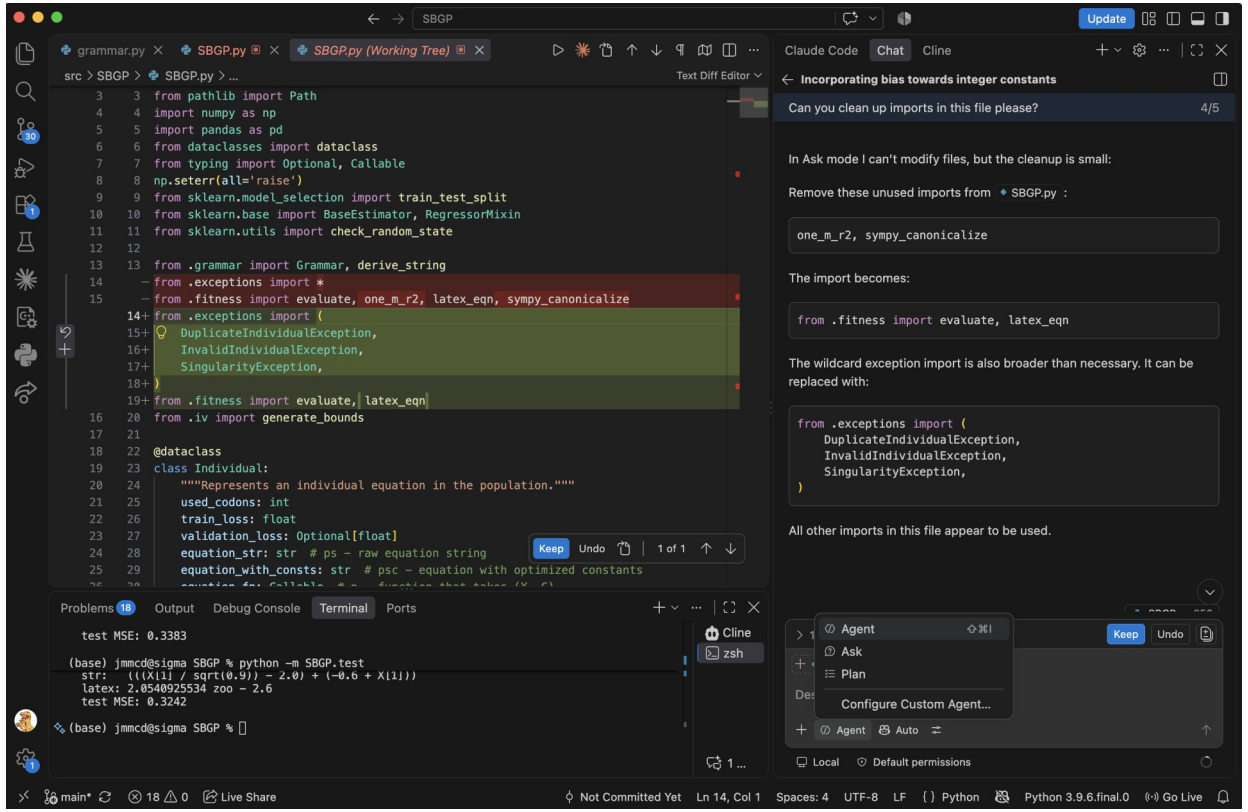
- Add your **@universityofgalway.ie** address to your GitHub account and **verify** it *before* you apply.
- Your GitHub **profile name** should match the name on your student card. A mismatch is the most common rejection.
- Have **proof of enrolment** ready to photograph: student card showing your name and the current academic year, or a dated enrolment letter.
- Apply while **on campus**, on the university network. Allow browser to access location if requested. Location is one of the signals used.
- If you are rejected, read the reason, fix it, and **reapply**.

Add and verify your university email first



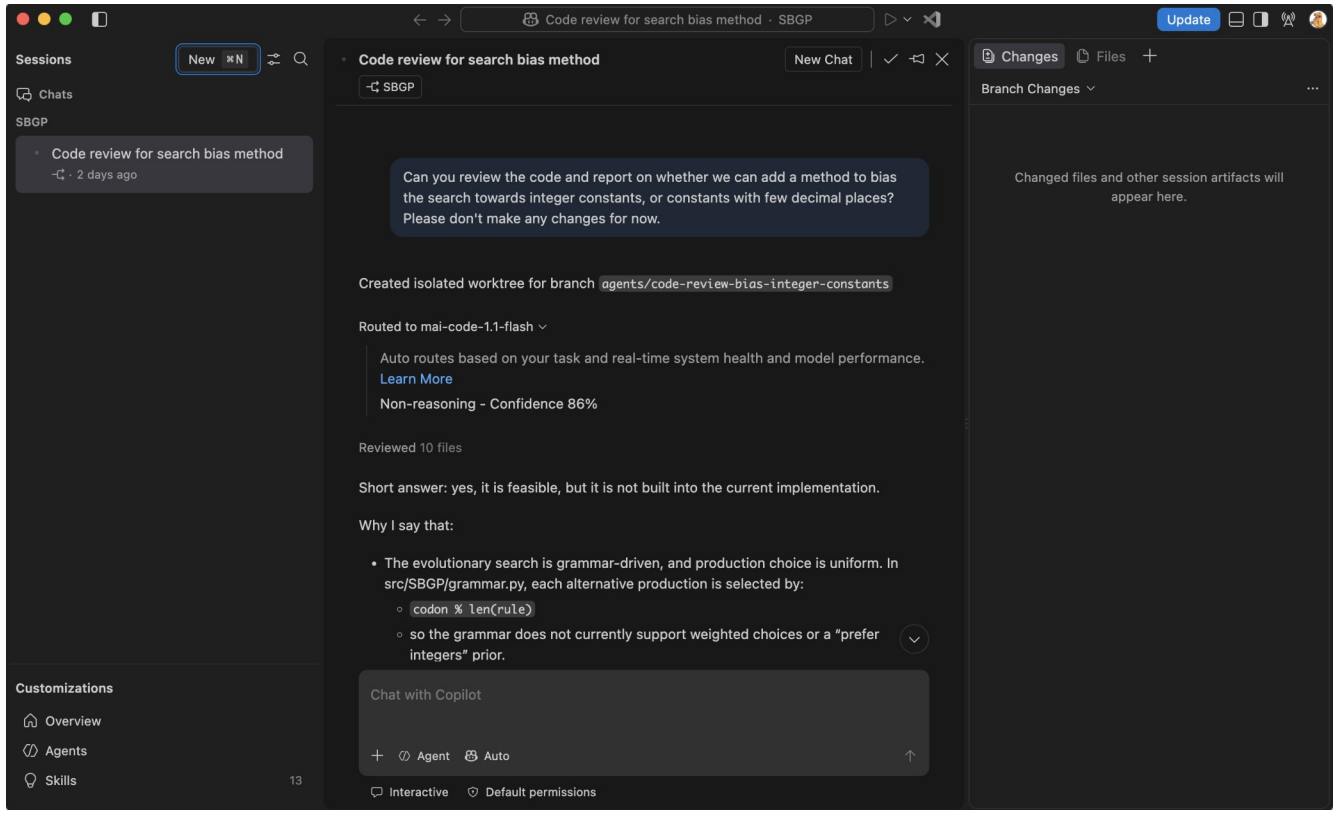
On GitHub: **Settings -> Emails -> Add email address**. Add your university address, then click the link in the mail GitHub sends you. Do this before applying. Once your licence is approved, sign in to VS Code with your GitHub account.

Using Copilot



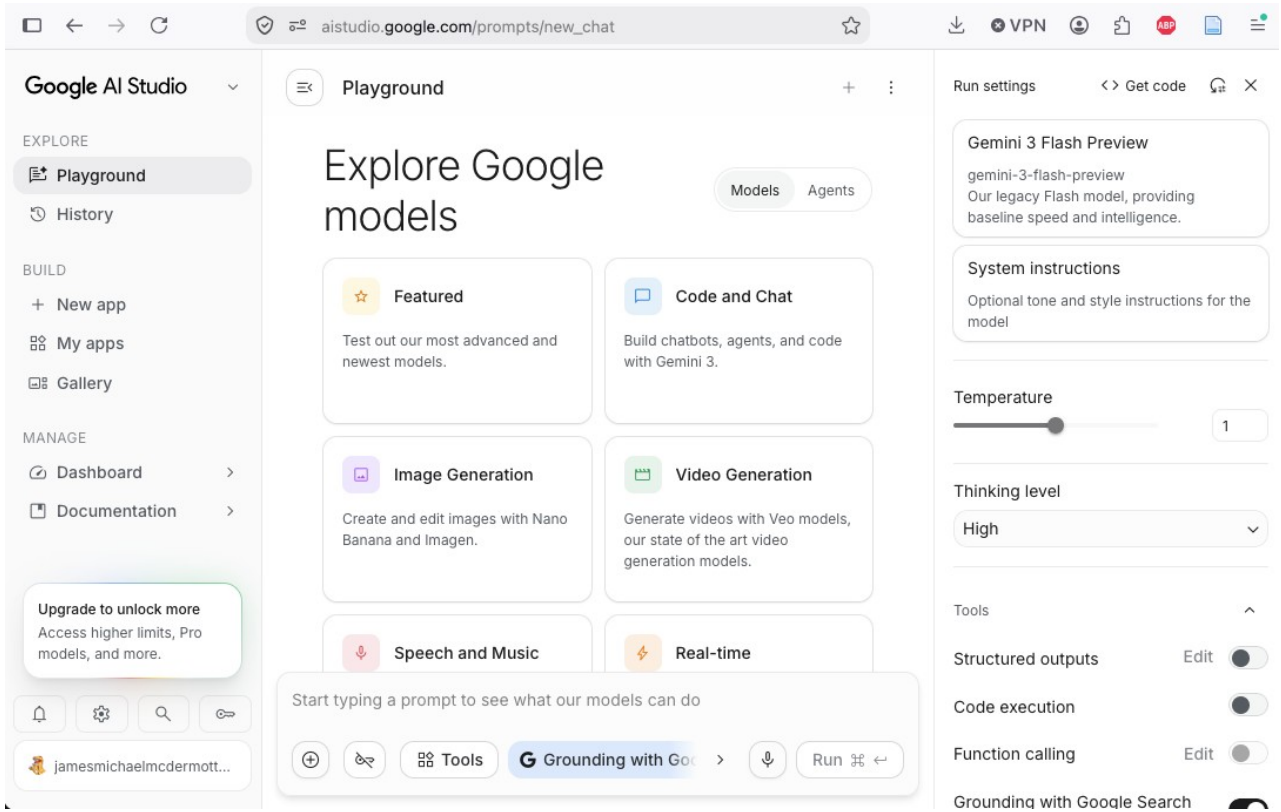
The Copilot panel opens on the right, named `Chat`. You can chat, read the "thinking", and ask for changes. Notice Agent versus Ask mode, and the Keep/Undo buttons. The Version Control panel (left) is also useful.

Using Agents



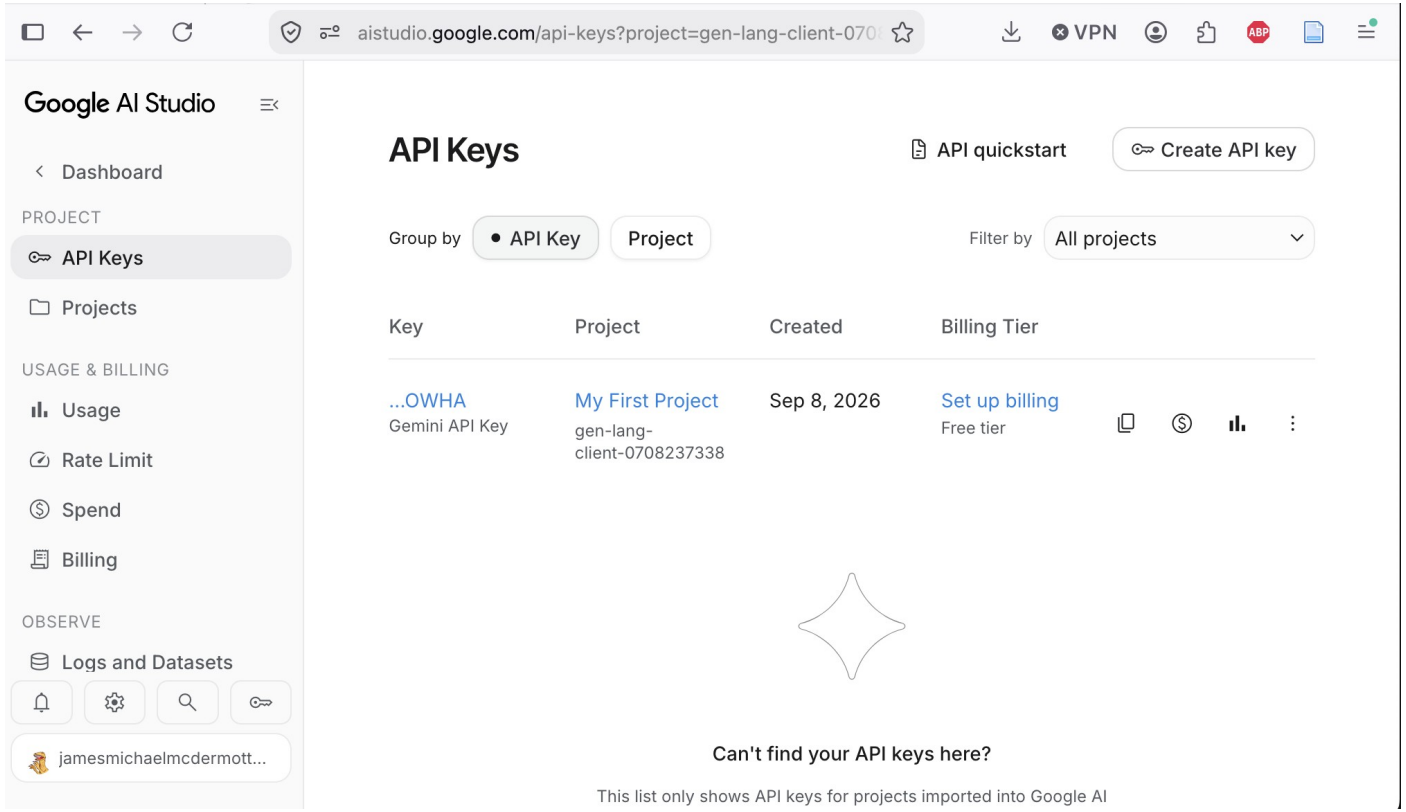
VS Code has an Agents icon in the top bar which opens this separate window for long-running agent interactions.

2. Google AI Studio



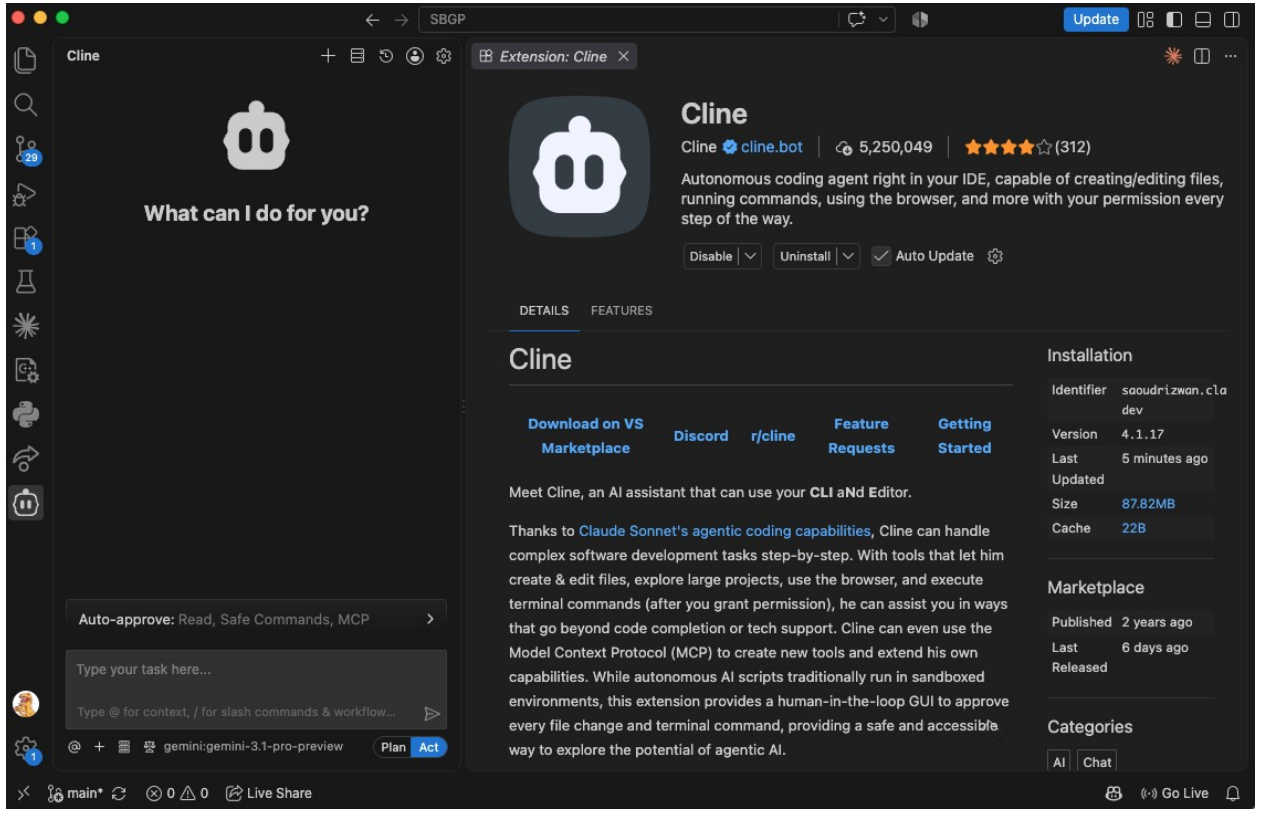
Option 2. Sign in at **aistudio.google.com** with any Google account. The Playground is the chat interface; we want the **API key** behind it.

Create an API key and copy it



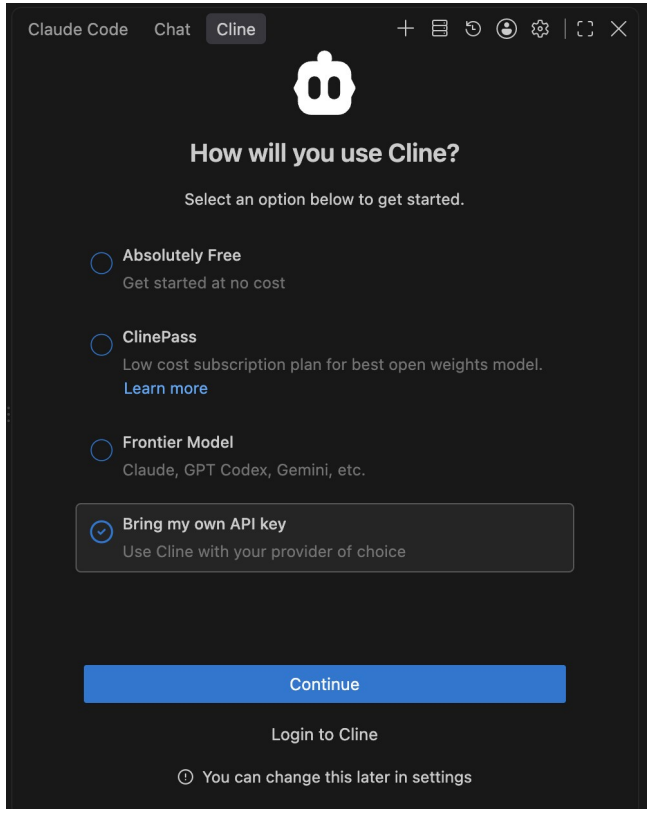
API Keys -> Create API key. A key must belong to a project; you have none, so let it make the placeholder **My First Project**. The key is then on the **Free tier**, with no billing set up, and the copy icon puts it on your clipboard. Treat it like a password: never paste it into a file you submit.

Install the Cline extension

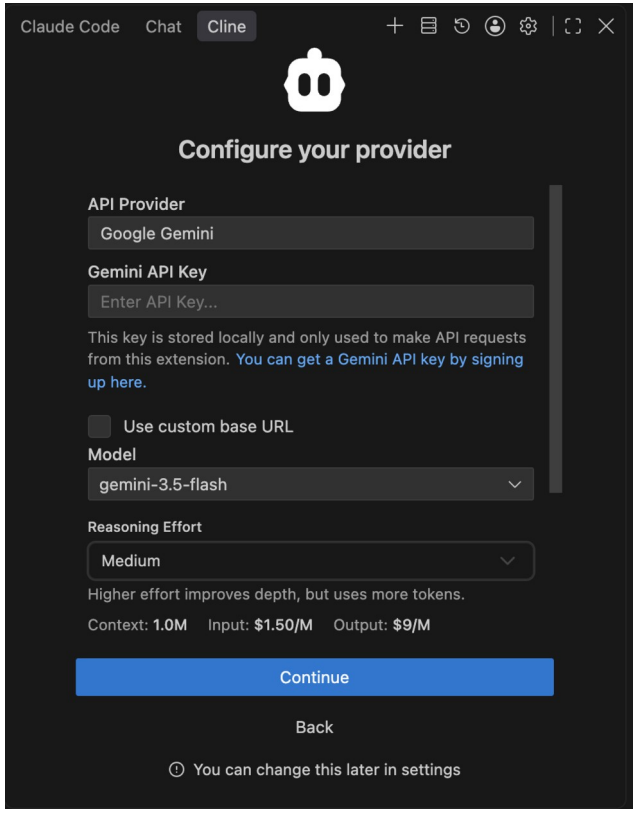


In VS Code, **Extensions**, search for **Cline**, and install it. It is not from Google: it is a general agent that talks to whichever model you give it a key for.

First run: your key, your model

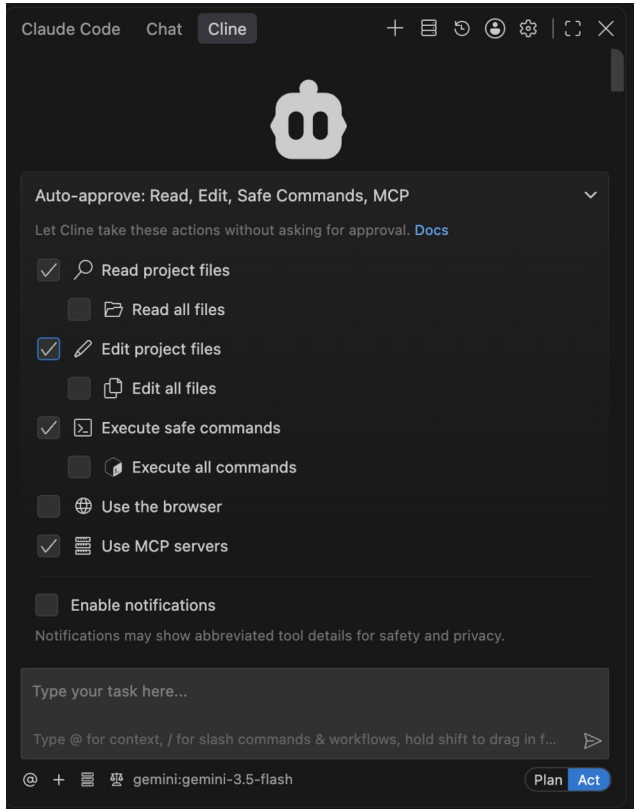


On first run, choose **Bring my own API key** — the others are Cline's own paid or limited plans.



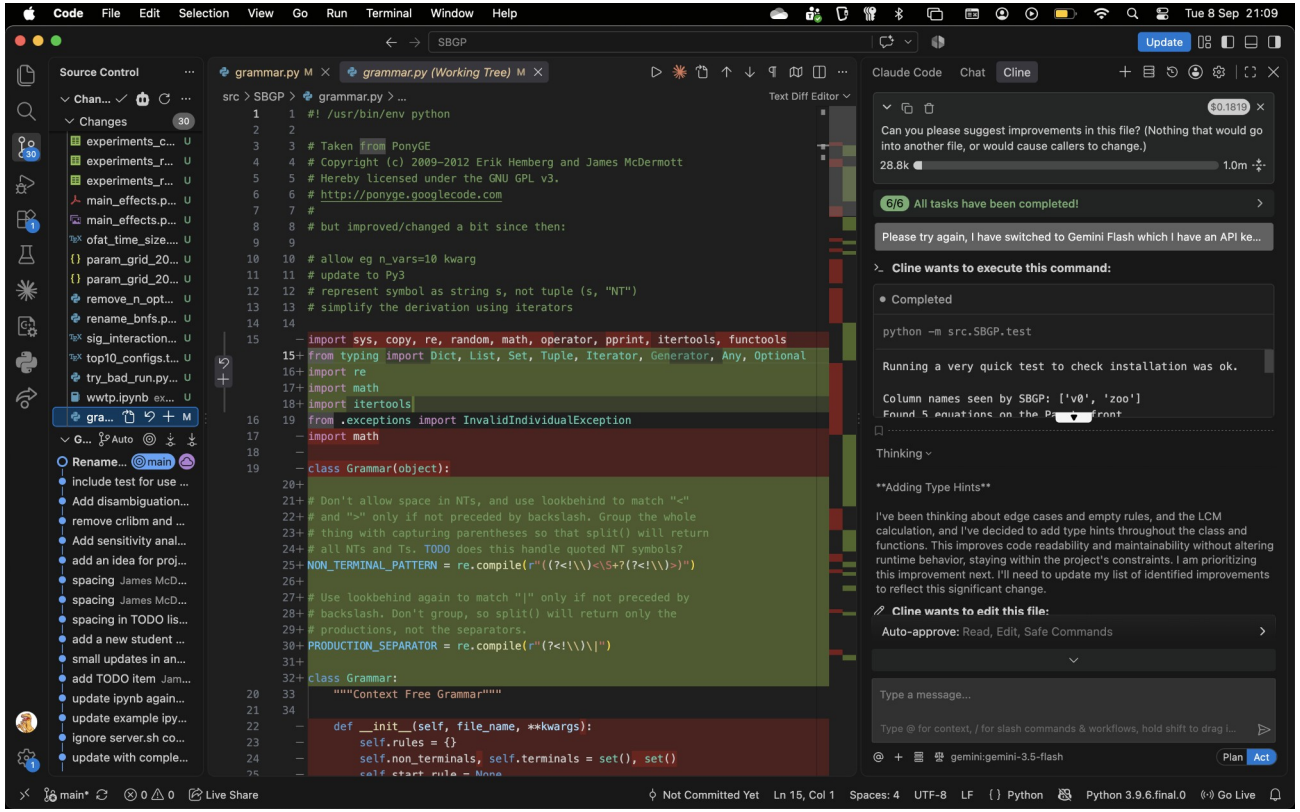
API Provider: Google Gemini, paste your key, pick a **Flash** model — small, fast, and the one with a usable free tier.

Auto-approve



You can configure what types of actions require manual approval.

Cline at work



Again we have a chat interface, some “thinking”, and a diff. Cline prints the notional **cost** of each finished task, which is a useful reality check even inside the free tier.